Lecture 26.03: Linear Regression with Gradient Descent — Step 1: Model & Parameter Initialization

📌 Executive Summary

First-principles breakdown of Step 1 in Gradient Descent: setting up the OOP class model architecture, parameter initialization strategies, and hyperparameter configuration.

🔬 Mathematical Formulation & Theory

1. The Optimization Paradigm

Finding the optimal parameters $\mathbf{w}^*$ and b^* analytically can be computationally prohibitive when the feature dimension d is vast. Gradient Descent provides an iterative first-order optimization algorithm that systematically maneuvers through parameter space toward the global minimum.

2. Parameter Initialization Strategies

Before optimization begins, parameters must be assigned initial starting values:

1. **Zero Initialization**:

$$\mathbf{w} = \mathbf{0}_d, \quad b = 0$$

Suitability: Because the Linear Regression cost surface is strictly convex (a quadratic bowl with zero local minima), zero initialization is completely safe and deterministic. It does not cause the symmetry-breaking pathologies found in deep neural networks.

2. **Small Gaussian Random Initialization**:

$$\mathbf{w} \sim \mathcal{N}\left(0, \sigma^2\right), \quad b = 0$$

Suitability: Useful when testing sensitivity to initial starting points or regularized optimization paths.

3. Hyperparameter Anatomy

The behavior of the gradient descent optimizer is controlled by crucial hyperparameters:

- **Learning Rate (α / `lr`)**: Step size scaling factor per iteration. Typical values: $0.001, 0.01, 0.1$.
- **Number of Iterations (`n_iter` / epochs)**: Total parameter update cycles executed across the training set.
- **Convergence Tolerance (`tol`)**: Early stopping condition if the gradient norm $\|\nabla J\| <$ tol.

4. OOP Class Architecture Skeleton

```
class LinearRegressionGD:
    def __init__(self, lr=0.01, n_iter=1000):
        self.lr = lr
        self.n_iter = n_iter
        self.weights = None
        self.bias = None
        self.loss_history = []
```

💻 Python & NumPy Implementation

```
import numpy as np

class LinearRegressionGDStep1:
    """Step 1: Boilerplate OOP Model Architecture with Parameter Initialization.""
    def __init__(self, lr=0.01, n_iter=1000):
        self.lr = lr
        self.n_iter = n_iter
        self.weights = None
        self.bias = None

    def _initialize_parameters(self, n_features: int):
        self.weights = np.zeros(n_features)
        self.bias = 0.0
        print(f"Initialized weights shape: {self.weights.shape}, bias: {self.bias}

model = LinearRegressionGDStep1(lr=0.01, n_iter=1000)
model._initialize_parameters(n_features=5)
print("Initial weights:", model.weights)
```

🎯 Key Takeaways & Best Practices

1. **First-Principles Transparency**: Building from scratch demystifies optimizer mechanics, memory layouts, and algorithmic failure modes.
2. **Vectorization vs Loops**: Always eliminate Python `for` loops across sample dimensions by relying on BLAS-accelerated NumPy array operations.
3. **Shape Integrity**: Maintain explicit assertions on input array ranks and shapes (`X.shape == (m, d)` , `y.shape == (m,)`) to prevent silent broadcasting bugs.